

AI 自動化拍攝 App 開發日誌

V1：電腦鏡頭 + 數位變焦 | V2：Sony 相機 + DJI 穩定器控制

專案名稱暫定：AutoCam Tracker

日期：2026-06-09

本文件整合目前專案討論結果、版本切分、C++ 套件最佳方案、V1 開發步驟、GitHub 管理規範與 V2 擴充方向。

1. 專案定位

本專案最終目標是開發一套 macOS 上運行的 C++ AI 自動化拍攝 App。系統會透過影像辨識偵測畫面中的車輛，讓使用者在介面上選擇要追蹤的車輛，並讓目標維持在畫面中心。

版本	核心定位	影像來源	控制方式
V1	軟體原型與追蹤流程驗證	MacBook Pro 內建鏡頭	數位變焦 / digital crop
V2	實體自動化拍攝	Sony 相機 / 擷取卡	DJI 穩定器 / CAN / 實體雲台控制

目前決策：V1 先不接 Sony SDK、不接 DJI、不接 CAN；先把「看到畫面 - 偵測車輛 - 點選目標 - 鎖定追蹤 - 數位裁切置中」這條核心流程做穩。

2. V1 版本定義：電腦鏡頭 + 數位變焦

V1 是可展示的 macOS C++ 原型 App，重點在 AI 偵測與單一目標追蹤邏輯。

- 使用 MacBook Pro 內建鏡頭作為 live view。
- 使用 YOLOv11 / YOLO11 系列模型進行車輛或物件偵測。
- UI 右側顯示偵測到的車輛縮圖。
- 使用者點擊車輛縮圖後，系統鎖定單一車輛追蹤。
- 系統計算目標 bbox 中心與畫面中心的 $error_x / error_y$ 。
- 透過 digital crop / 數位變焦模擬未來穩定器追蹤，讓輸出畫面中的目標維持在中心。
- V1 的數位追蹤功能在 V2 仍保留，作為無雲台模式、備援模式或後製裁切模式。

3. V2 版本定義：實體穩定器控制

V2 才加入 Sony 相機與 DJI 穩定器 / CAN 控制，把 V1 的追蹤控制邏輯接到實體設備上。

- 加入 Sony Camera Remote SDK / Camera Remote Command，用於相機連線、錄影、拍照、對焦與曝光控制。
- 加入 DJI RS SDK 或 CAN 控制模組，用於 pan / tilt 實體控制。
- 將 V1 的 FramingController 輸出，從 VirtualGimbalController 替換成 DjiRsController 或 CanGimbalController。
- 新增實體雲台安全限制：最大速度、最大角度、失控保護、手動接管。

4. 軟體套件最佳方案

以下是目前建議統一規格。原則是：V1 先用穩定、C++ 支援完整、macOS 好建置的方案；V2 再加入硬體 SDK。

功能區塊	最佳方案	理由	階段
------	------	----	----

V1 電腦鏡頭 + 數位變焦 / V2 穩定器控制

App 語言	C++20 (最低 C++17)	底層效能、硬體通訊、AI runtime 與跨平台擴充性較好。	V1
Build 系統	CMake	C++ 專案常用標準，Qt 官方文件也以 CMake 作為主要 build system。	V1
macOS UI	Qt 6	C++ 原生整合、macOS 支援完整，未來可移植 Windows / Linux。	V1
影像處理	OpenCV 4.x	可快速接 webcam、影片檔、frame 處理、bbox 繪製與裁切 thumbnail。	V1
AI 推論	ONNX Runtime C++	C/C++ API 完整，可載入 ONNX 模型，方便部署 YOLO。	V1
YOLO 模型	YOLOv11 / YOLO11 匯出 ONNX	先用現成 car / motorcycle 類別驗證流程，再換自訓模型。	V1
設定檔	nlohmann/json	C++ JSON 使用直覺，適合 default_config.json。	V1
Log	spdlog	C++ 高效能 logging，支援 console、file、rotating log。	V1
單元測試	GoogleTest	C++ 測試與 mock 常用方案，適合測 bbox、tracking、framing。	V1
版本管理	Git + GitHub	集中管理程式碼、Issue、PR、Code Review 與版本紀錄。	V1
相機控制	Sony Camera Remote SDK / Command	V2 用於 Sony 相機控制；V1 先用 MockCameraController。	V2
穩定器控制	DJI RS SDK 優先，CAN 第二階段	先走官方 SDK，CAN 作為需要更底層控制時的替代路線。	V2

5. 建議套件版本與使用方式

套件	建議	備註
C++	C++20，必要時降 C++17	不要使用太新的 C++23 功能，避免組員環境不一致。
Qt	Qt 6.x	用於 macOS interface、LiveViewWidget、VehicleListWidget。
OpenCV	OpenCV 4.x	V1 用 VideoCapture(0) 開 MacBook camera，也可讀影片檔做測試。
ONNX Runtime	最新穩定版 C++ API	推論層封裝成 OnnxRuntimeBackend，避免 UI 直接依賴 ONNX Runtime。
YOLO	YOLO11/YOLOv11 nano 或 small 起步	先重視 FPS 與流程穩定，再追求準確率。

spdlog	穩定版	所有模組統一使用，不要各自 cout debug。
GoogleTest	穩定版	針對 FramingController、SimpleTracker、DigitalCropController 寫測試。
nlohmann/json	穩定版	用於 config 讀寫，不需要引入過重設定系統。

6. V1 系統流程

```

MacBook Camera
↓
OpenCV VideoCapture
↓
YOLOv11 / YOLO11 Detection
↓
Detection List + Thumbnails
↓
UI Vehicle Selector
↓
TargetSelector / SimpleTracker
↓
FramingController: error_x / error_y
↓
DigitalCropController / VirtualGimbalController
↓
Centered Preview Output

```

7. 模組架構

```

AutoCamTracker/
├── app/                # AppController, AppState
├── ui/                 # MainWindow, LiveViewWidget, VehicleListWidget
├── video/              # IVideoSource, MacCameraSource, VideoFileSource
├── ai/                 # YoloDetector, OnnxRuntimeBackend, Detection
├── tracking/           # TargetSelector, SimpleTracker
├── control/            # FramingController, DeadZone, SmoothingFilter
├── virtual_gimbal/     # DigitalCropController, VirtualFrameStabilizer
├── camera/             # MockCameraController, V2 SonyCameraController
├── gimbal/             # VirtualGimbalController, V2 DjiRsController / CanGimbalController
├── config/             # default_config.json, ConfigManager
├── logging/            # Logger
├── tests/              # GoogleTest
├── docs/               # architecture, git rules, v1 requirements
└── CMakeLists.txt

```

8. 關鍵抽象介面

V1 一定要保留硬體抽象層，這是 V2 不重寫主程式的關鍵。

```

class IVideoSource {
public:
    virtual bool open() = 0;
    virtual bool read(cv::Mat& frame) = 0;
    virtual void close() = 0;
    virtual ~IVideoSource() = default;
};

struct Detection {
    int id;
    int classId;
    std::string label;
    float confidence;
    cv::Rect bbox;
    cv::Point center;
    cv::Mat thumbnail;
};

struct GimbalCommand {
    float panSpeed;
    float tiltSpeed;
    float rollSpeed;
    bool stop;
};

```

9. UI 功能規格

- 左側：Live View 主畫面。
- Live View 疊加：YOLO bbox、中心框、dead zone、目前追蹤目標框。
- 右側：Detected Vehicles Panel，顯示車輛縮圖。
- 點擊縮圖後鎖定單一目標。
- 下方狀態列顯示 FPS、inference time、tracking status、error_x、error_y、crop position。
- 設定面板可調整 confidence threshold、dead zone、smoothing、digital crop output size。

10. 數位變焦 / Digital Crop 設計

V1 沒有實體雲台，因此用 digital crop 模擬穩定器追蹤。原始畫面保留較大尺寸，輸出畫面使用較小 crop window。目標偏離中心時，crop window 往目標方向移動，讓輸出預覽中的目標回到中心。

- 目標在右邊：crop window 往右移。
- 目標在左邊：crop window 往左移。
- 目標在上方：crop window 往上移。
- 目標在下方：crop window 往下移。
- 加入 smoothing，避免畫面抖動。
- 加入邊界限制，避免 crop window 超出原始畫面。

11. Git / GitHub 開發規範

項目	規範
分支	main 為穩定版，develop 為整合開發版，feature/xxx 為功能分支，fix/xxx 為修 bug 分支。
PR	所有功能透過 Pull Request 合併，禁止直接 push main。
Commit	使用 feat / fix / refactor / docs / test / chore 前綴。
大檔案	YOLO 模型、影片素材、測試資料不要直接放普通 Git，必要時使用 Git LFS 或外部資料夾。
文件	docs/ 放 architecture.md、git_rules.md、v1_requirements.md、v2_expansion_plan.md。
Review	合併前至少一人 review，PR 描述需包含完成內容與測試方式。

12. V1 開發步驟

1. 建立 GitHub repository，設定 main / develop / feature branch 規範。
2. 建立 C++ / CMake / Qt 6 專案骨架，確認 macOS 可以 build。
3. 建立 IVideoSource interface 與 MacCameraSource。
4. 完成 LiveViewWidget，顯示 MacBook 鏡頭 live view。
5. 準備 YOLO11 / YOLOv11 ONNX 模型，建立 YoloDetector。
6. 完成 bbox 顯示與 Detection struct。
7. 從 bbox 裁切 thumbnail，建立 VehicleListWidget。
8. 點擊 thumbnail 後鎖定單一目標。
9. 建立 SimpleTracker，維持同一台車跨 frame 追蹤。
10. 建立 FramingController，計算 error_x / error_y 與 dead zone。
11. 建立 DigitalCropController，完成數位變焦 / 中心追蹤預覽。
12. 加入 default_config.json，讓參數可調整。
13. 加入 Log / Debug UI，顯示 FPS、inference time、tracking status、crop position。
14. 寫 README、build guide、architecture.md 與 V1 demo 流程。
15. 整合 V1 Demo，進行測試與修正。

13. V1 Definition of Done

- 專案已放上 GitHub，所有人使用 Git branch 開發。
- macOS 可以 build 並啟動 App。
- App 可以開啟 MacBook Pro 內建鏡頭並顯示 live view。
- YOLO11 / YOLOv11 可以在 C++ 中完成推論。
- Live view 可以顯示 bbox、中心框、dead zone。
- 右側可以顯示偵測到的車輛縮圖。
- 使用者可以點擊縮圖選擇追蹤目標。
- 系統可以維持追蹤單一目標，不會每幀亂跳。

- 系統可以計算目標與畫面中心偏移。
- Digital crop 可以讓輸出畫面中的目標接近中心。
- UI 可以顯示 FPS、tracking status、error_x、error_y。
- README 和 docs 有完整 build、執行、架構與 V2 擴充說明。

14. V1 暫不開發項目

- Sony SDK / Sony 相機控制
- DJI RS SDK
- CAN bus
- 實體穩定器控制
- 鏡頭對焦控制
- 車號 OCR
- DeepSORT / Re-ID
- 多目標自動導播
- 手機 App
- 雲端同步

15. V2 擴充方向

V1 模組	V2 替換 / 新增模組
MacCameraSource	SonyLiveViewSource / CaptureCardSource
MockCameraController	SonyCameraController
VirtualGimbalController	DjiRsController / CanGimbalController
DigitalCropController	保留，作為無雲台、備援、後製裁切模式
FramingController	保留，輸出改接實體 pan / tilt 控制
SimpleTracker	可升級 ByteTrack / DeepSORT / Re-ID

16. 團隊分工建議

角色	主要任務
架構 / GitHub 負責人	建立 repo、管理 branch、定義 interface、PR review、整合 develop。
UI 組	Qt interface、LiveWidget、VehicleListWidget、ControlPanel、StatusBar。
Video 組	IVideoSource、MacCameraSource、VideoFileSource、frame pipeline。
AI 組	YOLO ONNX 匯出、ONNX Runtime C++ 推論、bbox 後處理、thumbnail 裁切。
Tracking / Control 組	TargetSelector、SimpleTracker、FramingController、DeadZone、DigitalCropController。
測試 / 文件組	GoogleTest、README、build guide、architecture.md、demo script。

17. 風險與補項目

風險	補強方式
MacBook 鏡頭畫面與真實賽道差距大	V1 加入 VideoFileSource，可用賽道影片測試，不只靠 webcam。
YOLO 偵測框抖動	加入 dead zone、smoothing、lost timeout，並記錄 inference time。
多台車同框造成追蹤跳目標	V1 先用 SimpleTracker，V2 再升級 Re-ID 或 ByteTrack。
團隊 C++ 環境不一致	統一 CMake、README build 指令、版本表與 CI。
模型與影片檔過大	使用 Git LFS 或外部資料位置，避免 repo 膨脹。
V2 硬體接入風險高	V1 保留 IVideoSource / ICameraController / IGimbalController 抽象層。

18. 參考資料與官方文件

- Qt 6 Documentation: <https://doc.qt.io/qt-6/>
- Qt for macOS: <https://doc.qt.io/qt-6/macos.html>
- OpenCV VideoCapture Tutorial: https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
- ONNX Runtime API Docs: <https://onnxruntime.ai/docs/api/>
- ONNX Runtime C/C++ APIs: <https://onnxruntime.ai/docs/api/c/>
- Ultralytics YOLO11 Models: <https://docs.ultralytics.com/models/yolo11/>
- Ultralytics YOLO Export Mode: <https://docs.ultralytics.com/modes/export/>
- CMake Documentation: <https://cmake.org/>
- GoogleTest User Guide: <https://google.github.io/googletest/>
- spdlog GitHub: <https://github.com/gabime/spdlog>
- nlohmann/json GitHub: <https://github.com/nlohmann/json>
- Sony Camera Remote Toolkit: https://pro.sony/ue_US/digital-imaging/camera-remote-toolkit
- DJI RS SDK: <https://www.dji.com/rs-sdk>

19. 開發日誌結論

目前專案正式切分為 V1 與 V2：V1 使用電腦鏡頭與數位變焦先完成 AI 追蹤 App 原型；V2 再加入 Sony 相機、DJI 穩定器與 CAN 控制。

V1 的開發重點是把核心軟體流程做穩：影像輸入、YOLO 偵測、目標選擇、單一目標追蹤、畫面中心誤差計算、數位裁切置中。只要 V1 的流程穩定，V2 就可以把虛擬控制輸出替換成真實硬體控制，而不需要推倒重寫。

最重要原則：所有硬體相關功能都先做成抽象介面；V1 用虛擬模組，V2 再替換成 Sony / DJI / CAN 真實模組。